

Foundations of Reinforcement learning

[Reinforcement learning]

1.0 Content Block

Multi-objective reinforcement learning Multi-objective reinforcement learning (MORL) is a form of reinforcement learning concerned with conflicting alternatives. It is distinct from multi-objective optimization in that it is concerned with agents acting in environments.

2.0 Content Block

Fuzzy reinforcement learning By introducing fuzzy inference in reinforcement learning, approximating the state-action value function with fuzzy rules in continuous space becomes possible. The IF - THEN form of fuzzy rules make this approach suitable for expressing the results in a form close to natural language. Extending FRL with Fuzzy Rule Interpolation allows the use of reduced size sparse fuzzy rule-bases to emphasize cardinal rules (most important state-action values).

3.0 Content Block

A model of the environment is known, but an analytic solution is not available; Only a simulation model of the environment is given (the subject of simulation-based optimization); The only way to collect information about the environment is to interact with it. The first two of these problems could be considered planning problems (since some form of model is available), while the last one could be considered to be a genuine learning problem. However, reinforcement learning converts both planning problems to machine learning problems.

4.0 Content Block

$$V_{\pi}(s) = E \left[G \mid S_0 = s \right] = E \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right],$$

5.0 Content Block

$$G = \sum_{t=0}^{\infty} \gamma^t R_{t+1} = R_1 + \gamma R_2 + \gamma^2 R_3 + \dots,$$

6.0 Content Block

Associative reinforcement learning Associative reinforcement learning tasks combine facets of stochastic learning automata tasks and supervised learning pattern classification tasks. In associative reinforcement learning tasks, the learning system interacts in a closed loop with its environment.

7.0 Content Block

$R_a(s, s')$, the immediate reward after transitioning from s to s' under action a . The purpose of reinforcement learning is for the agent to learn an optimal (or near-optimal) policy that maximizes the reward function or other user-provided reinforcement signal that accumulates from immediate rewards. This is similar to processes that appear to occur in animal psychology. For example, biological brains are hardwired to interpret signals such as pain and hunger as negative reinforcements, and interpret pleasure and food intake as positive reinforcements. In some circumstances, animals learn to adopt behaviors that optimize these rewards. This suggests that animals are capable of reinforcement learning. A basic reinforcement learning agent interacts with its environment in discrete time steps. At each time step t , the agent receives the current state S_t

S_t and reward R_t . It then chooses an action A_t from the set of available actions, which is subsequently sent to the environment. The environment moves to a new state S_{t+1} and the reward R_{t+1} associated with the transition (S_t, A_t, S_{t+1}) is determined. The goal of a reinforcement learning agent is to learn a policy:

8.0 Content Block

In situation s perform action a . Receive a consequence situation s' . Compute state evaluation $v(s')$ of how good is to be in the consequence situation s' . Update crossbar memory $w'(a, s) = w(a, s) + v(s')$. Initial conditions of the memory are received as input from the genetic environment. It is a system with only one input (situation), and only one output (action, or behavior). Self-reinforcement (self-learning) was introduced in 1982 along with a neural network capable of self-reinforcement learning, named Crossbar Adaptive Array (CAA). The CAA computes, in a crossbar fashion, both decisions about actions and emotions (feelings) about consequence states. The system is driven by the interaction between cognition and emotion.

9.0 Content Block

For each possible policy, sample returns while following it. Choose the policy with the largest expected discounted return. One problem with this is that the number of policies can be large, or even infinite. Another is that the variance of the returns may be large, which requires many samples to accurately estimate the discounted return of each policy. These problems can be ameliorated if we assume some structure and allow samples generated from one policy to influence the estimates made for others. The two main approaches for achieving this are value function estimation and direct policy search.

10.0 Content Block

$\pi : S \times A \rightarrow [0, 1]$ $\pi(s, a) = \Pr(A_t = a \mid S_t = s)$

11.0 Content Block

The algorithm can be on-policy (it performs policy updates using trajectories sampled via the current policy) or off-policy. The action space may be discrete (e.g. the action space could be "going up", "going left", "going right", "going down", "stay") or continuous (e.g. moving the arm with a given angle). The state space may be discrete (e.g. the agent could be in a cell in a grid) or continuous (e.g. the agent could be located at a given position in the plane).

12.0 Content Block

Theory Both the asymptotic and finite-sample behaviors of most algorithms are well understood. Algorithms with provably (i.e. in a way that can be proved) good online performance (addressing the exploration issue) are known. Efficient exploration of Markov decision processes is given in Burnetas and Katehakis (1997). Finite-time performance bounds have also appeared for many algorithms, but these bounds are expected to be rather loose and thus more work is needed to better understand the relative advantages and limitations. For incremental algorithms, asymptotic convergence issues have been settled. Temporal-difference-based algorithms converge under a wider set of conditions than was previously possible (for example, when used with arbitrary, smooth function approximation).